

Table 1: Accuracy by Algorithmic Level, Averaged over Four Paraphrases

Method	GSM8K	CSQA	HumanEval	Summ.
Direct	[-]	[-]	[-]	[-]
Zero-shot CoT	[-]	[-]	[-]	[-]
Few-shot CoT	[-]	[-]	[-]	[-]
Self-consistency	[-]	[-]	[-]	[-]
Tree-of-thoughts	[-]	[-]	[-]	[-]
ReAct	[-]	[-]	[-]	[-]
Paraphrase range	[±]	[±]	[±]	[-]
AER	[-]	[-]	[-]	[-]

0.1 5.4 Protocol

Every cell is run with [s] seeds. Answers are extracted with a single regular expression shared by all conditions, so that differences in parsing cannot masquerade as differences in reasoning; parse failures are counted as errors and reported separately. Confidence intervals come from a paired bootstrap over items with 10,000 resamples, and comparisons across the algorithmic factor are corrected with the Holm–Bonferroni procedure. Variance components for (11) are estimated by restricted maximum likelihood with items treated as a random effect. Prompts, seeds, raw generations, and analysis scripts are released at [repository URL].

1 Results and Analysis

Section 6 reports measurements from the design of Section 5. Values marked in red must be replaced with the figures produced by your own runs.

1.1 6.1 Algorithmic versus lexical variation

Table 2 reports accuracy for each algorithmic level, averaged over the four paraphrases and reported at the reference decoding setting, with the spread across paraphrases given as a range. The comparison that bears on **H1** is between the vertical spread of the table—movement between cells of the taxonomy—and the horizontal spread within a row. On [task] the algorithmic factor moves accuracy by [Δ points] while paraphrasing moves it by [δ points], giving AER = [value]; the corresponding figures for the remaining tasks are [summarise]. **H1** is therefore [supported / partially supported / not supported].

1.2 6.2 Interaction with the decoding policy

Table 3 sweeps temperature and nucleus threshold for three algorithmic levels. Two patterns are of interest. First, the optimal temperature is not shared across methods: greedy CoT peaks at [T], whereas self-consistency requires $T > 0$ by construction and peaks at [T], which is the interaction predicted in **H2**. Second, the accuracy cost of a badly chosen temperature is [magnitude], which is [larger / smaller] than the gap between adjacent methods in Table 2. A technique reported at a single undocumented decoding

Table 2: Accuracy under Decoding Sweep (GSM8K)

top-p	T	Few-shot CoT	Self-cons.	ToT
1.0	0.0	[-]	n/a	[-]
1.0	0.3	[-]	[-]	[-]
1.0	0.7	[-]	[-]	[-]
1.0	1.0	[-]	[-]	[-]
0.9	0.3	[-]	[-]	[-]
0.9	0.7	[-]	[-]	[-]
0.9	1.0	[-]	[-]	[-]

Table 3: Distributional Effect Relative to Direct Answering (D_Φ in nats/token)

Condition	D_Φ	$\bar{\kappa}$	dist-2	s-BLEU
Paraphrase (mean)	[-]	[-]	[-]	[-]
Zero-shot CoT	[-]	[-]	[-]	[-]
Few-shot CoT	[-]	[-]	[-]	[-]
Retrieval prompt	[-]	[-]	[-]	[-]

setting is therefore [characterise the resulting ambiguity].

1.3 6.3 Distributional measurements

Table 4 reports the per-token divergence D_Φ between each prompt condition and the direct-answering baseline, the mean realised nucleus width $\bar{\kappa}$, and output diversity. The measurement that matters here is the relationship between D_Φ and accuracy: [state whether the two are monotonically related, and identify any condition with high divergence but no accuracy gain]. A condition of that kind is a prompt that moves the distribution without moving it usefully, which is precisely the case that raw benchmark numbers cannot distinguish from an inert prompt.

1.4 6.4 Cost

Table 5 converts accuracy into accuracy per thousand generated tokens. Ranking the methods by raw accuracy and by cost-adjusted accuracy gives [the same / a different] ordering; in particular [name the method whose ranking changes most]. Under a fixed token budget the preferred method is [method], which [does / does not] match the accuracy-optimal choice.

1.5 6.5 Error analysis

We hand-label $[n]$ errors per method into four categories: retrieval or knowledge failure, arithmetic slip, invalid reasoning step, and formatting or parse failure. The distribution over categories is [summarise]. Two predictions of Section 4 are checkable here. Proposition 3 implies that self-consistency should reduce arithmetic slips, which are high-variance and low-margin, while leaving invalid reasoning steps largely untouched; the observed change is [state]. The framing of ReAct as external conditioning implies that

Table 4: Accuracy per Thousand Generated Tokens (GSM8K)

Method	Acc.	Tok./item	Acc./1k tok
Direct	[-]	[-]	[-]
Zero-shot CoT	[-]	[-]	[-]
Few-shot CoT	[-]	[-]	[-]
Self-consistency	[-]	[-]	[-]
Tree-of-thoughts	[-]	[-]	[-]
ReAct	[-]	[-]	[-]

it should reduce knowledge failures specifically; the observed change is [state].

2 Discussion

The framework in Section 4 treats a prompt as a control input, and the experiments are constructed to test whether that view has empirical consequences. Three points deserve emphasis.

Reported gains carry a decoding configuration whether or not it is written down. Because trace-space and decoding-space operators interact, a chain-of-thought result at $T = 0$ and the same result at $T = 0.7$ are different measurements of different quantities. The practical recommendation that follows is narrow and cheap: report T , k , p , and m alongside every prompting result, and report the reference cell against which a gain is measured.

Divergence is necessary but not sufficient. A prompt that leaves $\hat{\Delta}_t \approx 0$ cannot change model behaviour, so a low D_Φ is decisive negative evidence about a proposed technique. The converse does not hold. High divergence with no accuracy gain is common, and it identifies prompts that redistribute probability mass without concentrating it on better answers—a failure mode that accuracy alone reports as a null result and that D_Φ reports as a mechanism.

Where a technique’s ceiling lies is determined by the operator class it belongs to. Proposition 3 shows that self-consistency converges to the marginal mode and can do no better, so it cannot rescue an item whose marginal mode is wrong; the fix has to come from prompt space, which changes the distribution, or from trace space with an external signal, as in ToT and ReAct. Reading Table 1 as a design space rather than a list makes this kind of diagnosis routine: identify which operator the failure implicates, then choose a technique from that class.

A consequence for practitioners is that the search for a better wording is usually the wrong first move. If the algorithmic effect ratio is high, the returns to rewording are bounded and the effort is better spent selecting an operator class and tuning its decoding parameters. If it is low, the reported technique is fragile and should be re-validated on the exact wording that will ship.

3 Limitations and Threats to Validity

Several limitations qualify these conclusions.

Model coverage. We evaluate open-weight models because token-level log-probabilities are required by (5). The largest proprietary models may respond differently, and prompting effects are known to vary with scale; our conclusions should be read as applying to the scales we test.

Paraphrase construction. The lexical factor depends on the claim that our four paraphrases preserve algorithmic structure. We control exemplar count, trace shape, and output format, but a paraphrase can still alter tokenisation boundaries or lexical overlap with pretraining data in ways we do not measure. This makes σ_{lex}^2 in (11) an upper bound on the purely lexical contribution, and hence AER a conservative estimate.

Task coverage. Four tasks cannot represent the space of LLM applications. Open-ended generation, multi-turn dialogue, and agentic workflows with long horizons are absent, and the last of these is where trace-space operators are most heavily used in practice.

Value estimation in ToT. The ToT results depend on a hand-specified value function. A better v would raise the ToT rows in every table, so those numbers are a lower bound on what the method can achieve rather than a measurement of the method itself.

Contamination. Public benchmarks may appear in pre-training corpora. Contamination would inflate absolute accuracies but should affect all cells of the design roughly equally, leaving the contrasts on which **H1** and **H2** depend comparatively robust.

Statistical power. With $[n]$ items per task and $[s]$ seeds, differences smaller than roughly $[\epsilon]$ points are not resolvable, and we do not claim them.

4 Conclusion

We have argued that prompt engineering techniques are best analysed as algorithmic operations on the conditional distribution an LLM defines over outputs, and we have given a taxonomy that classifies techniques by whether they act on the prompt, the decoding policy, or the reasoning trace. The formal treatment yields concrete statements: truncation operators bound per-step entropy in ways that differ between top- k and nucleus sampling, greedy chain-of-thought targets the joint mode while self-consistency targets the marginal mode, and the gap between the two accounts for the benefit of voting. A factorial experiment separating lexical from algorithmic variation supplies the corresponding measurements, and the algorithmic effect ratio summarises them in a single number.

The practical recommendation is that prompt engineering results be reported with the decoding configuration, the reference cell, and a divergence measurement attached, so that a reader can tell which operator produced a gain. Future work should extend the framework to multi-turn and agentic settings, where traces are long and the environment supplies part of the conditioning context, and should investigate whether the algorithmic effect ratio predicts how well a prompting technique transfers between models.

Acknowledgment

[Add acknowledgements and funding statement before submission.]

References

[1] T. Brown *et al.*, “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.*

(*NeurIPS*), vol. 33, 2020, pp. 1877–1901.

[2] S. Min, X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi, and L. Zettlemoyer, “Rethinking the role of demonstrations: What makes in-context learning work?,” in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, 2022, pp. 10748–10760.

[3] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 24824–24837.

[4] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[5] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 11809–11822.

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[7] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[8] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The curious case of neural text degeneration,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.

[9] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, “Large language models are zero-shot reasoners,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 22199–22213.

[10] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, “Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity,” in *Proc. Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2022, pp. 8086–8098.

[11] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, “Calibrate before use: Improving few-shot performance of language models,” in *Proc. Int.*

- Conf. Mach. Learn. (ICML)*, 2021, pp. 12697–12706.
- [12] A. Madaan *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 46534–46594.
- [13] A. Fan, M. Lewis, and Y. Dauphin, “Hierarchical neural story generation,” in *Proc. Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2018, pp. 889–898.
- [14] C. Meister, T. Pimentel, G. Wiher, and R. Cotterell, “Locally typical sampling,” *Trans. Assoc. Comput. Linguist.*, vol. 11, pp. 102–121, 2023.
- [15] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023.
- [16] S. Schulhoff *et al.*, “The prompt report: A systematic survey of prompting techniques,” 2024, arXiv:2406.06608. [Online]. Available: <https://arxiv.org/abs/2406.06608>
- [17] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, “Large language models are human-level prompt engineers,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [18] K. Cobbe *et al.*, “Training verifiers to solve math word problems,” 2021, arXiv:2110.14168. [Online]. Available: <https://arxiv.org/abs/2110.14168>
- [19] A. Talmor, J. Herzig, N. Lourie, and J. Berant, “CommonsenseQA: A question answering challenge targeting commonsense knowledge,” in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguist. (NAACL)*, 2019, pp. 4149–4158.
- [20] M. Chen *et al.*, “Evaluating large language models trained on code,” 2021, arXiv:2107.03374. [Online]. Available: <https://arxiv.org/abs/2107.03374>